

# Dokumentacja API (OpenAPI) – NoticeBoard

---

Poniższa dokumentacja opisuje wszystkie punkty końcowe (endpoints) dostępne w REST API projektu "Prosta Tablica Ogłoszeń".

**Wskazówka:** Po uruchomieniu aplikacji backendowej (domyślnie na porcie 8080), interaktywna dokumentacja Swagger UI jest dostępna pod adresem:  
`http://localhost:8080/swagger-ui.html` . Surowy plik OpenAPI JSON znajduje się pod `http://localhost:8080/v3/api-docs` .

---

## Moduł Uwierzytelniania (Authentication)

### 1. Rejestracja Użytkownika

- **Ścieżka:** `/api/auth/register`
- **Metoda:** `POST`
- **Dostęp:** Publiczny
- **Opis:** Tworzy nowe konto użytkownika w systemie. Hasło zostaje zahaszowane przed zapisaniem w bazie danych.

#### Request Body (application/json):

```
{
  "username": "user123",
  "password": "password123"
}
```

(Wymagania: `username` nie może być puste, `password` musi mieć minimum 4 znaki)

#### Responses:

- `200 OK` : Pomyślnie zarejestrowano użytkownika.
  - `400 Bad Request` : Błąd walidacji (np. za krótkie hasło) lub nazwa użytkownika jest już zajęta.
- 

### 2. Logowanie

- **Ścieżka:** `/api/auth/login`
- **Metoda:** `POST`
- **Dostęp:** Publiczny
- **Opis:** Weryfikuje poświadczenia i zwraca pomyślny status wraz z ustawieniem ciasteczka `HttpOnly` zawierającego token JWT.

#### Request Body (application/json):

```
{  
  "username": "user123",  
  "password": "password123"  
}
```

#### Responses:

- `200 OK` : Pomyślnie zalogowano. Serwer dodaje nagłówek `Set-Cookie: jwt={token}; HttpOnly; Path=/; Max-Age=86400` . Zwraca JSON z polem `username` .
  - `400 Bad Request` : Nieprawidłowa nazwa użytkownika lub hasło.
- 

### 3. Wylogowanie

- **Ścieżka:** `/api/auth/logout`
- **Metoda:** `POST`
- **Dostęp:** Publiczny (zazwyczaj używane przez zalogowanych)
- **Opis:** Czyści ciasteczko JWT, ustawiając jego ważność na 0.

#### Responses:

- `200 OK` : Wylogowano pomyślnie. Serwer ustawia pustą wartość nagłówka `Set-Cookie` dla tokenu.
- 

## Moduł Ogłoszeń (Notices)

### 4. Pobieranie listy ogłoszeń (Publiczne)

- **Ścieżka:** `/api/notices`
- **Metoda:** `GET`
- **Dostęp:** Publiczny

- **Opis:** Zwraca stronicowaną listę wszystkich ogłoszeń dodanych do systemu, wzbogaconą o linki nawigacyjne (HATEOAS).

### Parametry zapytania (Query Params):

- `page` (integer, opcjonalny): Numer strony (zaczynając od 0).
- `size` (integer, opcjonalny): Ilość elementów na stronie (domyślnie: 20).
- `sort` (string, opcjonalny): Sortowanie, np. `createdAt,desc`.

### Responses:

- **200 OK** : Zwraca obiekt `PagedModel` z danymi.

### Przykładowy Response:

```
{
  "_embedded": {
    "notices": [
      {
        "id": 1,
        "title": "Sprzedam samochód",
        "content": "Stan idealny, pierwszy właściciel.",
        "authorUsername": "janek",
        "createdAt": "2026-06-11T12:00:00",
        "_links": {
          "self": { "href": "http://localhost:8080/api/notices?page=0&size=20" }
        }
      }
    ]
  },
  "_links": {
    "self": { "href": "http://localhost:8080/api/notices?page=0&size=20" }
    "next": { "href": "http://localhost:8080/api/notices?page=1&size=20" }
  },
  "page": {
    "size": 20,
    "totalElements": 25,
    "totalPages": 2,
    "number": 0
  }
}
```

---

## 5. Pobieranie ogłoszeń autoryzowanego użytkownika

- **Ścieżka:** `/api/notices/my`
- **Metoda:** `GET`
- **Dostęp:** Zalogowani (Wymaga ważnego ciasteczka `jwt` )
- **Opis:** Identyfikuje użytkownika na podstawie tokenu i zwraca listę utworzonych wyłącznie przez niego ogłoszeń (z paginacją i HATEOAS).

#### Responses:

- `200 OK` : Pomyślnie zwrócono listę ogłoszeń w formacie `PagedModel` .
  - `403 Forbidden` : Brak ważnego tokenu JWT.
- 

## 6. Tworzenie ogłoszenia

- **Ścieżka:** `/api/notices`
- **Metoda:** `POST`
- **Dostęp:** Zalogowani (Wymaga ważnego ciasteczka `jwt` )
- **Opis:** Tworzy nowe ogłoszenie i przypisuje je do obecnie zalogowanego użytkownika.

#### Request Body (application/json):

```
{
  "title": "Kupię rower",
  "content": "Szukam roweru miejskiego, koła 28 cali."
}
```

#### Responses:

- `200 OK` : Ogłoszenie pomyślnie utworzone. Zwraca utworzone DTO wraz z linkami HATEOAS do powrotu do pełnej listy.
  - `400 Bad Request` : Brakujące pola (tytuł lub zawartość są puste).
  - `403 Forbidden` : Brak ważnego tokenu JWT.
- 

## 7. Usuwanie ogłoszenia

- **Ścieżka:** `/api/notices/{id}`
- **Metoda:** `DELETE`
- **Dostęp:** Zalogowani (Wymaga ważnego ciasteczka `jwt` )
- **Opis:** Usuwa ogłoszenie o podanym ID. Usunięcie zakończy się sukcesem wyłącznie wtedy, gdy żądający użytkownik jest faktycznym autorem tego ogłoszenia.

### Parametry ścieżki (Path Params):

- `id` (Long, wymagany): Unikalny identyfikator ogłoszenia.

### Responses:

- `204 No Content` : Ogłoszenie usunięte pomyślnie (brak zwracanego ciała).
- `400 Bad Request` : Ogłoszenie nie istnieje lub zalogowany użytkownik nie jest jego autorem (zabezpieczenie właścicielskie).
- `403 Forbidden` : Brak ważnego tokenu JWT.